

# 0.Geometric\_Algebra\_Introduction\_Kingdon

December 21, 2025

## 1 Geometric algebra for Computer Graphics

### 1.1 An all-in-one mathematical framework for virtual character rendering, animation, interpolation, deformation, cutting and tearing

## 2 Table of contents

1. State Of The Art
2. 3D CGA: A step beyond the dual-quaternions
3. The Euclidean Space
4. Conformal Geometric Algebra - Introduction
5. Vector Objects
  1. Points
  2. Spheres
  3. Planes
6. Clifford Implementation for Vector Objects
  1. Points
  2. Spheres
  3. Planes
7. Products in CGA
  1. Inner
  2. Geometric
    1. **Duality**
  3. Outer
8. Circles As Intersection Of Spheres
9. Lines As Intersection Of Planes
10. PointPairs As Intersection Of Spheres
11. Dual Objects
  1. Dual Sphere
  2. Dual Circle
  3. Dual Plane
  4. Dual Line
12. Complex Numbers Are Bivectors in CGA
13. **Translators**
14. **Rotors**
15. **Dilations**
16. **Motors**
17. Rotating lines and Circles

## 2.1 1. State of the art

Go to Top

The current state-of-the-art regarding virtual character rendering, animation, interpolation and deformation is dual quaternions 1. Using dual quaternions allows translations of points and rotations (with respect to any axis, even the ones not going through the origin) to be applied in a single framework and with great efficiency. It also helps avoid problems such as the gimbal lock that arise when using Euler angles and provides an elegant representation of rigid motions if seen as a rotation followed by a translation (often called *screw motion* 2).

Dual quaternions are the combination of dual numbers (introduced by Clifford) with quaternions (nicely written introduction in 4 and 3). This fusion allowed the implementation of translations under a single framework (see 3). Such a task was not possible before via the use of quaternions alone; it also required the use of matrices and/or vectors. This combined use of quaternions and matrices also required translating quaternions to vectors and vice-versa, which induced more memory usage and created major interpolation problems.

An algebra that contains dual quaternions as a sub-algebra is the so called Conformal Geometric Algebra (CGA). In CGA two new products are introduced: the *outer* and the *geometric* one. Entities such as spheres, planes, circles, points are easily seen both as objects and operators in this algebra (see 6 and 7). In this algebra, we can interpolate around any axis and transform all entities as objects and not simply point by point.

| Methodology                                  | Rotation | Translation   | Dilation      | Notes/ Issues                                                                 |
|----------------------------------------------|----------|---------------|---------------|-------------------------------------------------------------------------------|
| Euler Angles /<br>Transformation<br>Matrices |          | Matrices only | Matrices only | Gimbal lock,<br>problem when<br>interpolating                                 |
| Quaternions                                  |          |               |               | Interpolated only<br>at the origin, all<br>rotation axis<br>through origin    |
| Dual-<br>Quaternions                         |          |               |               | Interpolation only<br>at the origin,<br>transform points<br>only, no dilation |
| 3D PGA                                       |          |               |               | No dilation                                                                   |
| 3D CGA                                       |          |               |               | Interpolate around<br>any axis, transform<br>any entity                       |

[1] ( L. Kavan, S. Collins, J.Žára, and C. O’Sullivan. **Skinning with dual quaternions**. In Proceedings of the 2007 symposium on Interactive 3D graphics and games, pages 39–46.ACM, 2007) @@@ this section describes the existing bibliography with images

[2] (Daniilidis, K. (2016). **Hand-Eye Calibration Using Dual Quaternions**. The International Journal of Robotics Research, 18(3), 286–298. <http://doi.org/10.1177/02783649922066213>)

[3] (Kenwright, B. (2012). **A beginners guide to dual-quaternions: what they are, how they work, and how to use them for 3D character hierarchies.**)

- [4] (YB, Jia. **Dual Quaternions.** <http://web.cs.iastate.edu/~cs577/handouts/dual-quaternion.pdf>)
- [5] (H. Hadfield and J. Lasenby. Direct Linear Interpolation of Geometric Objects in Conformal Geometric Algebra)
- [6] (Leo Dorst, Daniel Fontijne, Stephen Mann. Geometric Algebra for Computer Science)
- [7] (Dietmar Hildenbrand. Foundations of Geometric Algebra Computing)

## 2.2 3D CGA: A step beyond the dual-quaternions

Go to Top

A major drawback of dual quaternions is the fact that they do not support scaling in a native way. This drawback can be overcome with the use of Geometric Algebra and specifically Conformal Geometric Algebra (see [7,8]). CGA contains both quaternion and dual-quaternion algebras as sub-algebras with the main difference that (dual) quaternion multiplication is replaced by the so-called *geometric product*. Therefore, after translating all dual quaternions to the respective elements of CGA, called *multivectors*, one can take advantage of all tools provided by quaternions. The translation is naive as we simply have to translate all base elements of one algebra to another, as shown in the following table:

| Element of dual quaternion | Respective element in CGA |
|----------------------------|---------------------------|
| 1                          | 1                         |
| $i$                        | $e_1 \wedge e_2$          |
| $j$                        | $-e_1 \wedge e_3$         |
| $k$                        | $e_2 \wedge e_3$          |
| $i\epsilon$                | $e_3 e_\infty$            |
| $j\epsilon$                | $e_2 e_\infty$            |
| $k\epsilon$                | $e_1 e_\infty$            |
| $i\epsilon$                | $e_1 e_2 e_3 e_\infty$    |

**Example** The dual quaternion  $d = 3 + 2i + \epsilon(4j + k)$  corresponds in CGA to the multivector  $D = 3 + 2e_1 \wedge e_2 + 4e_2 \wedge e_\infty + e_1 \wedge e_\infty$ .

In CGA, entities like points, planes, spheres, circles and lines can be easily represented as multivectors, i.e., linear combinations of the base elements  $\{1, e_1, e_2, e_3, e_0, e_\infty, e_1 \wedge e_2, \dots, e_1 \wedge e_2 \wedge e_3, \dots, I := e_1 \wedge e_2 \wedge e_3 \wedge e_0 \wedge e_\infty\}$ . Rotors and rigid movements that corresponded to quaternions and dual quaternions respectively can also be represented as multivectors. Most importantly, dilations can also be represented as multivectors. Ultimately, if  $D$  represents a deformation (rotor, motor, dilation or any combination of these three) in CGA and  $E$  is any of the aforementioned entities in CGA, the new entity  $E' := DE\tilde{D}$ , where  $\tilde{D}$  is the *reverse* of  $D$ , is the entity that results after applying  $D$  to  $E$ . Note that the operation implied between  $D$ ,  $E$  and  $\tilde{D}$  is the geometric product of multivectors. In the reverse of a multivector the blades are with reversed order of their outer product components, for instance the reverse of  $1 + e_1 \wedge e_2$  is equal to  $1 + e_2 \wedge e_1$  or  $1 - e_1 \wedge e_2$ .

Being able to apply rotations, translations and dilation in a uniform way in so many entities, and not just points, is an extraordinary tool. It saves us from the need to translate our objects back and forth from dual quaternionic form to matrices and provides a straightforward way to apply

any deformation in a way that easily encapsulates all corresponding information. Furthermore, interpolations can be easily defined and also include interpolation on axis that do not go through the origin, which is a major restriction in dual quaternions.

[7] Dorst et al. **Geometric algebra for computer science: an object-oriented approach to geometry. (2010).** Geometric algebra for computer science: an object-oriented approach to geometry.

[8] Doran et al. **Geometric algebra for physicists. (2003).** Geometric algebra for physicists.

The following table summarizes our GA algorithms for virtual character animation interpolation and deformation

| Algorithms  | Symbolic (Python modules)               | Performance (C++/GPU) |
|-------------|-----------------------------------------|-----------------------|
| QUAT - LBS  | Quaternions.py / Quaternions + GAOnline | glGA                  |
| DQUAT - DQS | DualQuaternions.py + GAOnline/pyganja   | glGA                  |
| EGA - EGS   | Clifford + GAOnline/pyganja             | glGA                  |
| CGA - CGS   | Clifford + GAOnline/pyganja             | glGA                  |

*Table Abbreviations:* - **QUAT LBS : Quaternion - Linear Blend Skinning**, N. Magnenat-Thalmann, R. Laperrire, and D. Thalmann. Joint-dependent local deformations for hand animation and object grasping. In *Proceedings on Graphics interface'88*. Citeseer, 1988 - **DQUAT DQS : Dual quaternion - Dual Quaternion Skinning**, L. Kavan, S. Collins, J.Žára, and C. O'Sullivan. Skinning with dual quaternions. In *Proceedings of the 2007 symposium on Interactive 3D graphics and games*, pages 39–46. ACM, 2007 - **EGA EGS : Euclidian Geometric Algenra - Euclidean Geometric Skinning**, Papagiannakis G. Geometric algebra rotors for skinned character animation blending. Technical Brief, ACM SIGGRAPH ASIA 2013, Hong Kong, November 2013. September 2013:1-6. - **CGA CGS : Conformal Geometric Algebra - Conformal Geometric Skinning**, Papaefthymiou M, Hildenbrand D, Papagiannakis G. An inclusive Conformal Geometric Algebra GPU animation interpolation and deformation algorithm. *Vis Comput.* 2016;32(6-8):1-9. doi:10.1007/s00371-016-1270-8.

### 2.3 3. The Euclidean Space

Go to Top

In the 3-Dimensional Euclidean space, let  $\{e_1, e_2, e_3\}$  denote the orthonormal basis of  $\mathbb{R}_3$ . Then it is known that

- A point  $p$  is represented as  $(x_p, y_p, z_p)$ .
- A line  $\ell$  is represented as  $p + s \cdot \vec{t} = (x_p, y_p, z_p) + s(x_t, y_t, z_t)$ ,  $s \in \mathbb{R}$  where  $p$  is a point on the line and  $t$  is a vector parallel to  $\ell$ .
- A plane  $\Pi$  is represented as  $\{p \in \mathbb{R}_3 : \vec{n} \cdot (p - p_0) = 0\}$  or equivalently  $\{(x_p, y_p, z_p) \in \mathbb{R}_3 : x_n \cdot x_p + y_n \cdot y_p + z_n \cdot z_p = x_n \cdot x_0 + y_n \cdot y_0 + z_n \cdot z_0\}$ , where  $\vec{n} = (x_n, y_n, z_n)$  is a vector perpendicular to  $\Pi$  and  $p_0 = (x_0, y_0, z_0)$  is a point on the plane.
- A sphere  $S$  is the locus of points  $p = (x_p, y_p, z_p) \in \mathbb{R}_3$  such that  $(x_p - x_C)^2 + (y_p - y_C)^2 + (z_p - z_C)^2 = r^2$ , where  $C = (x_C, y_C, z_C)$  is the center of the sphere and  $r$  is its radius

- A circle in 3 dimensions is difficult to be explicitly defined algebraically. It can be viewed algebraically as the intersection of a sphere with either a sphere or a plane.
- A rotation in 3 dimensions can be described via explicitly via the axis and the angle of rotation or implicitly via Euler Angles and Matrices.
- A dilation in 3 dimensions can be described only for vectors/points as a multiplication of a vector with a scale factor.
- A translation in 3 dimensions can be described and implemented, depending on the object that is translated. For example, to translate a plane we simply translate the point  $p_0$  on the plane but not the perpendicular vector, whereas translating a circle requires (complete) reevaluation of the algebraic expression.

It is apparent that the Euclidean space, introduced a naive way of perceiving and implementing various geometric entities. However, as objects become more complicated, their representation and the ability to transform them becomes non-trivial. Our need to simplify translations, rotations and dilations of such objects led to the so-called *Conformal Geometric Algebra*.

## 2.4 4. Conformal Geometric Algebra - Introduction

Go to Top

Conformal Geometric Algebra (CGA) is a projective geometry tool which allows conformal transformations to be implemented with rotations. To do this, the original space, in our case  $\mathbb{R}^3$ , is extended by two dimensions, one of positive signature  $e_+$  and one of negative signature  $e_-$ . Thus, if we started with  $G_p$ , the conformal space is  $G_{p+1,1}$ , i.e., in our case  $\mathbb{R}_{4,1}$ .

It is convenient to define a *null* basis given by the original vectors  $e_1, e_2, e_3$  of  $\mathbb{R}^3$  and

$$e_o = \frac{1}{2}(e_- - e_+), \quad e_\infty = e_- + e_+.$$

## 2.5 5. Vector objects in $\mathbb{R}_{4,1}$

Go to Top

A generic vector  $Y$  in the 3D-CGA is a linear combination of these vectors

$$Y = y_1 e_1 + y_2 e_2 + y_3 e_3 + y_\infty e_\infty + y_o e_o, \quad y_i \in \mathbb{R}.$$

Note that since we are in a projection space there is an equivalence relation, i.e., the objects  $Y$  and  $Z$  are equivalent iff there is a  $\lambda \in \mathbb{R}$  such that  $y_i = \lambda z_i$  for all  $i$ . Due to this equivalence, we usually assume, without loss of generality, that the coordinate of  $e_o$  is either 0 or 1.

In CGA, points, spheres and planes are easily expressed as vector objects.

### 2.5.1 5.A Points

Go to Top

A point  $x = (x_1, x_2, x_3) = x_1 e_1 + x_2 e_2 + x_3 e_3$  of the original space is *up-projected* into a conformal vector  $X$ , defined as

$$\begin{aligned}
X &= x + \frac{1}{2}x^2e_\infty + e_o \\
&= x_1e_1 + x_2e_2 + x_3e_3 + \frac{1}{2}(x_1^2 + x_2^2 + x_3^2)e_\infty + e_o
\end{aligned}$$

### 2.5.2 5.B Spheres

Go to Top

A sphere  $s$  of the original space, centered at  $x = (x_1, x_2, x_3)$  with radius  $r$  is *up-projected* into a conformal vector  $S$ , defined as

$$\begin{aligned}
S &= X - \frac{1}{2}r^2e_\infty \\
&= x_1e_1 + x_2e_2 + x_3e_3 + \frac{1}{2}(x_1^2 + x_2^2 + x_3^2 - r^2)e_\infty + e_o,
\end{aligned}$$

where  $X$  is the image of  $x$  in CGA.

### 2.5.3 5.C Planes

Go to Top

A plane  $\pi$  of the original space, perpendicular to the **normal** vector  $\vec{n} = (n_1, n_2, n_3)$ , such that  $d(\pi, O) = d$  (where  $d(\cdot, \cdot)$  is the Euclidean distance) is *up-projected* into a conformal vector  $\Pi$ , defined as

$$\begin{aligned}
\Pi &= n + de_\infty \\
&= n_1e_1 + n_2e_2 + n_3e_3 + de_\infty.
\end{aligned}$$

## 2.6 6. Vector Objects - Clifford implementation

Go to Top

```
[59]: import sys
import warnings
warnings.filterwarnings('ignore')
# !pip install kingdon pyganja
```

```
[60]: from pyganja import *
```

```
[61]: import numpy as np
from kingdon import Algebra
from pyganja import *

# =====
# 1. SETUP ALGEBRA
# =====
# Create Algebra (4, 1) for Conformal Geometric Algebra (CGA)
```

```

# Signature (4, 1) means 4 positive squared basis vectors and 1 negative.
alg = Algebra(4, 1)

# Inject basis blades into local namespace (e1, e2, e3, e4, e5, etc.)
locals().update(alg.blades)

# =====
# 2. DEFINE BASIS CONVENTIONS
# =====
# Kingdon (4,1) has basis vectors e1, e2, e3, e4, e5.
# Standard CGA convention maps them as follows:
# e1, e2, e3 -> Euclidean basis
# e4 -> e+ (positive extra dimension)
# e5 -> e- (negative extra dimension)
ep = e4
en = e5

# Define Null Basis vectors (origin and infinity)
# einf (infinity) = e- + e+
# eo (origin) = 0.5 * (e- - e+)
einf = en + ep
eo = 0.5 * (en - ep)

# =====
# 3. HELPER FUNCTIONS
# =====

def get_scalar(mv):
    """
    Helper to extract the scalar value from a Kingdon MultiVector.
    If the MV is a scalar, returns its float value.
    """
    # In Kingdon, we can access the scalar part (grade 0) using index 0
    # if it's stored as an array, or we can try casting.
    # A robust way for a pure scalar MV is accessing the first element of its
    ↪ values.
    try:
        return float(mv)
    except:
        # If it's a MultiVector, try accessing the scalar component (index 0
        ↪ usually corresponds to scalar '1')
        # However, Kingdon's internal storage might vary.
        # Let's assume it's a scalar result from an inner product.
        if hasattr(mv, 'values'):
            # Assuming the first value is the scalar one for a scalar MV
            return mv.values()[0]
        return 0.0

```

```

def up(x):
    """
    Up-projects a 3D Euclidean vector 'x' to a 5D Conformal Point 'P'.
    Formula:  $P = x + 0.5 * x^2 * e_{inf} + e_o$ 
    """
    # (x|x) is the inner product (squared magnitude)
    return x + 0.5 * (x|x) * e_inf + e_o

def down(P):
    """
    Down-projects a 5D Conformal Point 'P' back to a 3D Euclidean vector.
    """
    # 1. Normalize the point so that  $P \mid e_{inf} = -1$ 
    # This handles the homogeneous nature of CGA points.
    scale = -1.0 / (P | e_inf)
    P_norm = P * scale

    # 2. Extract the Euclidean vector part.
    # We project P_norm onto the e1, e2, e3 basis.
    # (P_norm | e1) gives the coefficient for e1, etc.
    # We use get_scalar to ensure we get a float.
    x = get_scalar(P_norm | e1) * e1
    y = get_scalar(P_norm | e2) * e2
    z = get_scalar(P_norm | e3) * e3
    return x + y + z

def homo(P):
    """
    Homogenizes a CGA point P so that its weight ( $P \mid e_{inf}$ ) is -1.
    """
    return P / (-P | e_inf)

def random_sphere():
    """Creates a random sphere in CGA."""
    x = np.random.random(3)
    r = np.random.random()
    center = x[0]*e1 + x[1]*e2 + x[2]*e3
    # Sphere = Center_Point - 0.5 * r^2 * e_inf
    return up(center) - 0.5 * r**2 * e_inf

def random_plane():
    """Creates a random plane in CGA."""
    n = np.random.random(3)
    d = np.random.random()
    vector = n[0]*e1 + n[1]*e2 + n[2]*e3
    # Plane = Normal_Vector + distance * e_inf

```



```
return vector + d * einf
```

Let us inspect stuff:

```
[62]: print(alg)
      print("Basis:", alg.blades)
```

```
Algebra(p=4, q=1, r=0, signature=array([ 1,  1,  1,  1, -1]))
Basis: BladeDict(algebra=Algebra(p=4, q=1, r=0, signature=array([ 1,  1,  1,  1, -1])), lazy=False)
```

It contains the following:

- **ep** - positive basis vector, i.e., ep is  $e_+$ .
- **en** - negative basis vector, i.e., en is  $e_-$ .
- **eo** - zero vector of null basis, i.e., eo is  $e_o$ .
- **einf** - infinity vector of null basis, i.e., einf is  $e_\infty$ .
- **E0** - minkowski bivector ( $=\text{einf}\wedge\text{eo}$ )
- **up()** - function to up-project a vector from GA to CGA
- **down()** - function to down-project a vector from CGA to GA
- **homo()** - function to homogenize a CGA vector

```
[63]: locals().update(alg.blades)
```

## 2.6.1 6.A Implementation of Points

Go to Top

```
[64]: # Create a random 3D point using numpy
      x = np.random.rand(3)

      # Construct the vector in the Geometric Algebra (R3 subspace)
      # e1, e2, e3 are the basis vectors for x, y, z
      point = x[0]*e1 + x[1]*e2 + x[2]*e3

      print('point in R3= ', point)
```

```
point in R3=  0.17   + 0.92   + 0.0112
```

```
[65]: # Project the 3D point 'point' into the Conformal Geometric Algebra (CGA) space
      # This creates a null vector representation of the point
      P = up(point)

      print('point in CGA= ', P)
      print('P*P = ', P*P) # Should be 0 (null vector)
```

```
point in CGA=  0.17   + 0.92   + 0.0112   + -0.0624   + 0.938
P*P =  1.11e-16
```

```
[112]: # Visualize the point P
# We use a scale factor to ensure it's visible
alg.graph([P], scale=0.1, color=0xFF0000)
```

```
[112]: GraphWidget(cayley=[['1', 'e1', 'e2', 'e3', 'e4', 'e5', 'e12', 'e13', 'e14',
'e15', 'e23', 'e24', 'e25', 'e34'...
```

```
[67]: # Create a sphere S
# A sphere is defined as  $S = P - 0.5 * r^2 * \text{einf}$ 
# where  $P$  is the center point (in CGA) and  $r$  is the radius
P1 = up(0.5*e1) # Center at (0.5, 0, 0)
r = 0.5 # Radius 0.5
S = P1 - 0.5 * r**2 * einf

print('Sphere S = ', S)
print('S*S = ', S*S) # Should be  $r^2$ 
```

```
Sphere S =  0.5    + -0.5    + 0.5
S*S =  0.25
```

A non-homogeneous vector in CGA is a vector where the coefficient of  $e_o$  is  $\neq \{0,1\}$ .

```
[68]: # Create a plane pi
# A plane is defined as  $pi = n + d * \text{einf}$ 
# where  $n$  is the normal vector and  $d$  is the distance from the origin
n = e1 + e2 + e3
n = n / np.sqrt((n|n).values()[0]) # Normalize the normal vector
d = 0.5 # Distance from origin
pi = n + d * einf

print('Plane pi = ', pi)
print('pi*pi = ', pi*pi) # Should be 1 (normalized plane)
```

```
Plane pi =  0.577    + 0.577    + 0.577    + 0.5    + 0.5
pi*pi =  1.0
```

```
[69]: alg.graph([pi, P])
```

```
[69]: GraphWidget(cayley=[['1', 'e1', 'e2', 'e3', 'e4', 'e5', 'e12', 'e13', 'e14',
'e15', 'e23', 'e24', 'e25', 'e34'...
```

The function `homo` returns the respective homogeneous vector of the input one.

```
[70]: # Create a circle C
# A circle is the intersection of a sphere and a plane
#  $C = S \wedge pi$ 
C = S ^ pi

print('Circle C = ', C)
```

```
print('C*C = ', C*C)
```

```
Circle C =  0.289    + 0.289    + 0.539    + -0.0387    + 0.289    + -0.289
           + 0.289    + -0.289    + -0.5
C*C =  -0.205 + -5.55e-17    + -5.55e-17
```

## 2.6.2 6.B Implementation of Spheres

Go to Top

```
[71]: # Create a point pair Pp
      # A point pair is the intersection of three spheres (or a circle and a sphere)
      # Pp = S1 ^ S2 ^ S3
      P2 = up(-0.5*e1)
      S2 = P2 - 0.5 * r**2 * e1f
      Pp = S ^ S2 ^ pi

      print('Point Pair Pp = ', Pp)
      print('Pp*Pp = ', Pp*Pp)
```

```
Point Pair Pp =  0.289    + -0.289    + 0.289    + -0.289    + -0.5
Pp*Pp =  0.25
```

```
[72]: # Create a line L
      # A line is the intersection of two planes
      # L = pi1 ^ pi2
      n2 = e1 - e2
      n2 = n2 / np.sqrt((n2|n2).values()[0])
      pi2 = n2 + 0.2 * e1f
      L = pi ^ pi2

      print('Line L = ', L)
      print('L*L = ', L*L)
```

```
Line L =  -0.816    + -0.408    + -0.238    + -0.238    + 0.408    + 0.469
           + 0.469    + 0.115    + 0.115
L*L =  -1.0 + 2.78e-17    + 6.94e-18    + 6.94e-18
```

```
[73]: alg.graph([S])
```

```
[73]: GraphWidget(cayley=[['1', 'e1', 'e2', 'e3', 'e4', 'e5', 'e12', 'e13', 'e14',
'e15', 'e23', 'e24', 'e25', 'e34'...
```

```
[74]: # Create a flat point Fp
      # A flat point is the intersection of a line and a plane (or 3 planes)
      # Fp = L ^ pi3
      n3 = e3
      pi3 = n3 + 0.0 * e1f
      Fp = L ^ pi3
```

```
print('Flat Point Fp = ', Fp)
print('Fp*Fp = ', Fp*Fp)
```

Flat Point Fp = -0.816 + 0.238 + 0.238 + -0.469 + -0.469

Fp\*Fp = -0.667

### 2.6.3 6.C Implementation of Planes

Go to Top

```
[ ]: # Visualize the objects using pyganja
# We use alg.graph which handles the conversion from Kingdon to Ganja.js
↳ automatically
```

```
objects = [P, S, pi, C, Pp, L, Fp]
alg.graph(objects, scale=0.05)
```

<IPython.core.display.Javascript object>

```
[76]: # Define a plane
# We need a normal vector 'vector' and a distance 'd'
vector = e1 + e2 + e3
vector = vector / np.sqrt(get_scalar(vector|vector)) # Normalize
d = 1.0
plane = vector + d * einf
print('the vector of the plane in R4,1 = ', plane)
```

the vector of the plane in R4,1 = 0.577 + 0.577 + 0.577 + 1.0 + 1.0

```
[77]: # Visualize the plane
# In CGA, a grade-1 vector represents a sphere or plane in the Inner Product
↳ Null Space (IPNS)
sc = GanjaScene()
sc.add_objects([get_coefs(plane)])
draw(sc, scale=0.05)
```

<IPython.core.display.Javascript object>

**Note** that you cannot down-project a vector of CGA whose coordinate of  $e_o$  is 0.

## 2.7 7. Products in $\mathbb{R}_{4,1}$

Go to Top

There are 3 major products in  $\mathbb{R}_{4,1}$ : the inner, the outer and the geometric. Each of these products is initially defined among the vectors of either base of the space, i.e.,  $\{e_1, e_2, e_3, e_-, e_+, e_o, e_\infty\}$ . Their definition is then extended to included any vector (or multi-vector) of the space.

### 2.7.1 7.A Inner Product

Go to Top

The inner product (denoted by  $|$ ) of the basis elements is defined as follows:  $* e_i|e_j = \delta_{ij}$  for  $i, j \in \{1, 2, 3, +\}$   $* e_-|e_- = -1$   $* e_-|e_j = 0$  for  $j \in \{1, 2, 3, +\}$   $* e_o|e_o = e_\infty|e_\infty = 0$   $* e_o|e_\infty = -1$   $* e_i|e_j = 0$  for  $i \in \{1, 2, 3, +\}$  and  $j \in \{o, \infty\}$

```
[78]: # Inner product of e3 and einf
# e3 is Euclidean, einf is null infinity. They are orthogonal.
res = e3 | einf
print('e3 | einf = ', res)
```

e3 | einf = 0

```
[79]: # Inner product of eo and einf
# eo (origin) and einf (infinity) are not orthogonal. Their inner product is -1.
res = eo | einf
print('eo | einf = ', res)
```

eo | einf = -1.0

### 2.7.2 7.B Geometric Product

Go to Top

More details in later Sections, if needed.

```
[80]: # Geometric product of e1 and e2
# Since e1 and e2 are orthogonal, e1*e2 = e1~e2 = e12 (bivector)
res = e1 * e2
print('e1 * e2 = ', res)
```

e1 \* e2 = 1

One of the most common use of the geometric product is the evaluation of the dual of an object - multivector.

### 2.7.3 Duality

Go to Top

The dual operator  $*$  is used to calculate the dual  $O^*$  of a multivector  $O$  where

$$O^* := -O \cdot I,$$

where  $I := e_1 \wedge e_2 \wedge e_3 \wedge e_+ \wedge e_- = e_1 \wedge e_2 \wedge e_3 \wedge e_\infty \wedge e_o$  and  $\cdot$  is the geometric product. The function `dual()` can be used to directly evaluate the dual of an object.

```
[81]: # Demonstrate duality
# Create random objects
O1 = random_sphere()
```

```

O2 = random_plane()
# Intersection (meet) via geometric product (if one is dual?) or just product
# Here we just multiply them to get a multivector O
O = O1 * O2
print('An object: \nO=', O)
print('\nIt\'s dual: \nO*=', O.dual())

# Define the Pseudoscalar I
# In CGA (4,1), I = e1^e2^e3^e4^e5
I = e1^e2^e3^e4^e5

# Verify duality convention: O* = -O . I (or similar depending on convention)
# Kingdon's dual() usually implements O * I_inv
# Let's check what it does
print('\n-O * I = ', -O * I)
# Note: The exact sign depends on the dimension and signature.

```

An object:

```

O= 0.439 + -0.0287    + -0.136    + 0.189    + -0.223    + -0.276    +
0.432    + -0.501    + 0.23     + -0.226    + -0.0648

```

It's dual:

```

O*= 0.0648    + -0.226    + 0.23     + 0.501    + -0.432    + -0.276
    + -0.223    + 0.189    + 0.136    + -0.0287    + -0.439

```

```

-O * I = 0.0648    + -0.226    + 0.23     + 0.501    + -0.432    +
-0.276    + -0.223    + 0.189    + 0.136    + -0.0287    + -0.439

```

**Remark** It holds that

$$(O^*)^* = -O$$

```

[82]: # Verify double duality
# (O*)* = -O
res = O.dual().dual()
print('(O*)* = ', res)
print('-O = ', -O)
# assert(res == -O) # Floating point issues might prevent exact equality

```

```

(O*)* = -0.439 + 0.0287    + 0.136    + -0.189    + 0.223    + 0.276    +
-0.432    + 0.501    + -0.23     + 0.226    + 0.0648
-O = -0.439 + 0.0287    + 0.136    + -0.189    + 0.223    + 0.276    +
-0.432    + 0.501    + -0.23     + 0.226    + 0.0648

```

## 2.7.4 7.C Outer Product

Go to Top

The outer product of  $n$  different vectors of the basis form  $n$ -vectors (in bibliography these are called

$n$ -blades, or blades of degree  $n$ ) for  $n \in \{2, 3, 4, 5\}$ . Swapping the order of two consecutive vectors changes the sign of the corresponding blade. A sum of blades of different degrees is what we call a **multivector**.

In Python, a blade is always printed as an outer product of the original  $e_i$  vectors,  $i \in \{1, 2, 3, 4, 5\}$ , where  $e_4 \equiv e_+$  and  $e_5 \equiv e_-$ . The vector  $e_1 \wedge e_2 \wedge e_3$  is abbreviated as  $e123$ ,  $e12$  corresponds to  $e_1 \wedge e_2$  and so on.

```
[83]: # Outer product of e1 and e2
      res = e1 ^ e2
      print('e1 ^ e2 = ', res)
```

e1 ^ e2 = 1

```
[84]: # Outer product of parallel vectors is 0
      res = e1 ^ e1
      print('e1 ^ e1 = ', res)
```

e1 ^ e1 = 0

```
[85]: # Outer product of e1 and e3
      res = e1 ^ e3
      print('e1 ^ e3 = ', res)
```

e1 ^ e3 = 1

```
[86]: # Outer product is anti-commutative
      res = e2 ^ e1
      print('e2 ^ e1 = ', res)
```

e2 ^ e1 = -1

The most important property of the outer product is that it represents, in a sense and in certain occasions, the *intersection* of (specific) objects. For example, a circle can be described (in CGA) as the intersection-outer product of two spheres and a line as the intersection of two planes. There is also a new geometric object called **Point Pair**, which is a set of two points. A point pair can be described in CGA as the intersection of 3 spheres.

## 2.8 8. Implementation of Circles as Intersection of Spheres

Go to Top

```
[87]: # Contraction (Inner product)
      # (e1^e2) | e2 = e1 (left contraction)
      res = (e1^e2) | e2
      print('(e1^e2) | e2 = ', res)
```

(e1^e2) | e2 = 1

```
[88]: # Contraction with orthogonal vector
      # (e1^e2) | e3 = 0
      res = (e1^e2) | e3
```

```
print('(e1^e2) | e3 = ', res)
```

$(e1^e2) | e3 = 0$

```
[89]: # Inner product of bivectors
# (e1^e2) | (e2^e3)
# = e1 . (e2 . (e2^e3)) = e1 . (e3) = 0?
# Or using identity: (A^B)|(C^D) = (A.D)(B.C) - (A.C)(B.D)
# (e1.e3)(e2.e2) - (e1.e2)(e2.e3) = 0*1 - 0*0 = 0
res = (e1^e2) | (e2^e3)
print('(e1^e2) | (e2^e3) = ', res)
```

$(e1^e2) | (e2^e3) = 0$

## 2.9 9. Implementation of Lines as Intersection of Planes

Go to Top

```
[90]: # Geometric product of bivectors
# (e1^e2) * (e2^e3) = e1 e2 e2 e3 = e1 (1) e3 = e1 e3 = e1^e3
res = (e1^e2) * (e2^e3)
print('(e1^e2) * (e2^e3) = ', res)
```

$(e1^e2) * (e2^e3) = 1$

```
[91]: # Outer product of bivectors
# (e1^e2) ^ (e2^e3)
# Since they share e2, the outer product is 0.
res = (e1^e2) ^ (e2^e3)
print('(e1^e2) ^ (e2^e3) = ', res)
```

$(e1^e2) ^ (e2^e3) = 0$

```
[92]: # Outer product of bivector and vector
# (e1^e2) ^ e3 = e1^e2^e3 = e123 (trivector/pseudoscalar of R3)
res = (e1^e2) ^ e3
print('(e1^e2) ^ e3 = ', res)
```

$(e1^e2) ^ e3 = 1$

## 2.10 10. Implementation of PointPairs as Intersection of Spheres

Go to Top

```
[93]: # Geometric product of parallel vectors
# e1 * e1 = e1.e1 + e1^e1 = 1 + 0 = 1
res = e1 * e1
print('e1 * e1 = ', res)
```

$e1 * e1 = 1$



```
[94]: # Geometric product of orthogonal vectors
#  $e_1 * e_2 = e_1 \cdot e_2 + e_1 \wedge e_2 = 0 + e_{12} = e_{12}$ 
res = e1 * e2
print('e1 * e2 = ', res)
```

$e_1 * e_2 = 1$

```
[95]: # Geometric product of orthogonal vectors
#  $e_1 * e_3 = e_{13}$ 
res = e1 * e3
print('e1 * e3 = ', res)
```

$e_1 * e_3 = 1$

## 2.11 11. Dual objects representation

[Go to Top](#)

In this section, we provide an alternative way to define spheres, planes, circles, lines and point pairs via their dual representation.

### 2.11.1 11.A Implementation of a dual Sphere

[Go to Top](#)

If  $S$  is a sphere, then we can define its dual  $S^*$  as the outer product of 4 points lying on its boundary.

```
[96]: # Associativity of geometric product
#  $e_1 * e_1 * e_2 = (e_1 * e_1) * e_2 = 1 * e_2 = e_2$ 
res = e1 * e1 * e2
print('e1 * e1 * e2 = ', res)
```

$e_1 * e_1 * e_2 = 1$

Note that in the code above, we used the function `normal()` to actually create objects of magnitude (norm of some kind)  $= \pm 1$ . Since we are in a projection space the objects  $O$ ,  $-5O$  and  $3O$  correspond to the same object with different magnitude/orientation.

So two objects can be the same but not necessarily have the same magnitude/orientation! For example, the objects  $e_1 + e_\infty$ ,  $2e_1 + 2e_\infty$  and  $-5e_1 - 5e_\infty$  all correspond to the same plane!

### 2.11.2 11.B Implementation of a dual Circle

[Go to Top](#)

If  $C$  is a sphere, then we can define its dual  $C^*$  as the outer product of 3 points lying on its boundary.

```
[97]: # Geometric product of bivectors
#  $(e_1 * e_2) * (e_2 * e_3) = e_1 * (e_2 * e_2) * e_3 = e_1 * 1 * e_3 = e_{13}$ 
res = (e1 * e2) * (e2 * e3)
```

```
print('(e1*e2) * (e2*e3) = ', res)
```

```
(e1*e2) * (e2*e3) = 1
```

```
[98]: # Geometric product of bivector and vector
# (e1*e2) * e3 = e123 (pseudoscalar of R3)
res = (e1*e2) * e3
print('(e1*e2) * e3 = ', res)
```

```
(e1*e2) * e3 = 1
```

### 2.11.3 11.C Implementation of a dual Plane

Go to Top

If  $P$  is a plane, then we can define its dual  $P^*$  can be seen as the dual of an infinite sphere, i.e., the outer product of 3 points lying on the plane and  $e_\infty$ .

```
[99]: # Inverse of a vector
# For a vector v, v^-1 = v / |v|^2. If |v|=1, v^-1 = v.
# e1 is unit vector, so e1^-1 = e1
res = e1.inv()
print('e1.inv() = ', res)
```

```
e1.inv() = 1.0
```

```
[100]: # Inverse of a bivector
# (e1*e2)^-1 = e2^-1 * e1^-1 = e2 * e1 = -e12
res = (e1*e2).inv()
print('(e1*e2).inv() = ', res)
```

```
(e1*e2).inv() = -1.0
```

### 2.11.4 11.D Implementation of a dual Line

Go to Top

If  $L$  is a plane, then we can define its dual  $L^*$  can be seen as the dual of an infinite circle, i.e., the outer product of 2 points lying on the line and  $e_\infty$ .

```
[101]: # Inverse of a trivector
# (e1*e2*e3)^-1 = e3*e2*e1 = -e123
res = (e1*e2*e3).inv()
print('(e1*e2*e3).inv() = ', res)
```

```
(e1*e2*e3).inv() = -1.0
```

```
[102]: # Inverse of the pseudoscalar I
# I = e1^e2^e3^e4^e5
# I*I = -1 (in R4,1) ? Let's check.
res = I.inv()
print('I.inv() = ', res)
```

```
print('I * I.inv() = ', I * res)
```

```
I.inv() = -1.0
I * I.inv() = 1.0
```

## 2.12 12. Complex Numbers are bivectors in CGA

Go to Top

Given an orthonormal  $\{e_1, e_2, e_3\}$  basis of a 3-dimensional GA, we can define the quantities  $i, j$  and  $k$  as follows:

$$i = e_1 \wedge e_2, \quad j = e_1 \wedge e_3, \quad k = e_2 \wedge e_3.$$

Given their definition, the following properties hold

$$\begin{aligned} i^2 &= (e_1 \wedge e_2)^2 = e_1 e_2 \underbrace{e_1 e_2}_{-e_2 e_1} = -e_1 \underbrace{e_2 e_2}_1 e_1 = -\underbrace{e_1 e_1}_1 = -1 \\ j^2 &= (e_1 \wedge e_3)^2 = e_1 e_3 \underbrace{e_1 e_3}_{-e_3 e_1} = -e_1 \underbrace{e_3 e_3}_1 e_1 = -\underbrace{e_1 e_1}_1 = -1 \\ k^2 &= (e_2 \wedge e_3)^2 = e_2 e_3 \underbrace{e_2 e_3}_{-e_3 e_2} = -e_2 \underbrace{e_3 e_3}_1 e_2 = -\underbrace{e_2 e_2}_1 = -1 \\ ij &= (e_1 \wedge e_2)(e_1 \wedge e_3) = e_1 \underbrace{e_2 e_1}_{-e_1 e_2} e_3 = -\underbrace{e_1 e_1}_1 e_2 e_3 = -e_2 \wedge e_3 = -k \\ jk &= (e_1 \wedge e_3)(e_2 \wedge e_3) = e_1 \underbrace{e_3 e_2}_{-e_2 e_3} e_3 = -e_1 e_2 \underbrace{e_3 e_3}_1 = -e_1 \wedge e_2 = -i \\ ki &= (e_2 \wedge e_3)(e_1 \wedge e_2) = \underbrace{e_2 e_3}_{-e_3 e_2} \underbrace{e_1 e_2}_{-e_2 e_1} = e_3 \underbrace{e_2 e_2}_1 e_1 = -e_1 \wedge e_3 = -j \\ ijk &= ii = -1 \end{aligned}$$

It also holds that

$$ji = k, \quad kj = i \quad \text{and} \quad ik = j.$$

The properties of  $i, j$  and  $k$  defined above coincide with the properties of the imaginary units  $i, j$  and  $k$  that are introduced during the definition of a quaternion. Consequently, we have proven that the quaternion algebra ( $\mathbb{H}$ ) is contained in GA (which is contained in CGA).

Based on the following remarks, a quaternion

$$Q = q_o + q_1 i + q_2 j + q_3 k,$$

where  $q_i \in \mathbb{R}$ , is an *even* multivector, i.e., a sum of blades of even degree (0 or 2).

## 2.13 13. Translators

Go to Top

Here is the Translation formula in CGA :

$$T = \exp(-\frac{1}{2}te_{\infty}) = 1 - \frac{1}{2}te_{\infty},$$

where  $t = t_1e_1 + t_2e_2 + t_3e_3$  is the translation vector. The a rigid object  $O$  is translated to  $O_2$  via the operation  $O_2 = TO\tilde{T}$ .

## 2.14 14. Rotors

Go to Top

Here is the Rotation formula in CGA :

$$R = \exp(-b\frac{\phi}{2}) = \exp(-I_3u\frac{\phi}{2}) = \cos(\frac{\phi}{2}) - uI_3\sin(\frac{\phi}{2}),$$

where  $\phi$  is the angle of the rotation  $b$  is the normalized plane of the rotation  $u$  is the normalized axis of the rotation  $I_3 = e_1e_2e_3$

The a rigid object  $O$  is rotated to  $O_2$  via the operation  $O_2 = RO\tilde{R}$ .

Note that the Rotation formula can be applied in both GA and CGA.

```
[103]: # Rotors example
# Rotation by angle phi around axis u
phi = 2*np.pi/3 # rotation angle (120 degrees)
u = e1 + e2 + e3 # vector (1,1,1)
print('u before norm:', u)
u_sq = get_scalar(u|u)
print('u|u:', u_sq)
u = u / np.sqrt(u_sq) # normalized vector
print('u normalized:', u)

I3 = e1*e2*e3 # Pseudoscalar of R3 (Euclidean)
print('I3:', I3)

# Rotor R = exp(-phi/2 * B) where B is the bivector of the rotation plane
# The rotation plane is dual to the axis u in R3: B = I3 * u
B = I3 * u
print('B (Rotation Plane):', B)

# Kingdon supports exp via alg.exp or math functions if enabled
# Let's try using the library's exp function if available, or check if np.exp
↳works
# If np.exp returns [], it means it didn't work.
try:
    R = alg.exp(-B * (phi/2))
```

```

except:
    # Fallback: use Taylor series or manual formula
    #  $R = \cos(\phi/2) - B * \sin(\phi/2)$ 
    R = np.cos(phi/2) - B * np.sin(phi/2)

print ('Rotor = ', R)

O = 5*e1 # our object: the point (5,0,0) in R3
print ('Original Point-Object = ', O)

# Apply rotor:  $O_2 = R * O * \sim R$ 
# Check if  $\sim R$  works
try:
    R_rev = ~R
except:
    R_rev = R.reverse() # Explicit method call

O_2 = R * O * R_rev
print( 'Rotated Point-object = ', O_2) # Should be (0,5,0)

# Rotate a sphere
S = up(e1 + 3*e2) - 0.5 * 1**2 * e1f
S_2 = R * S * R_rev
print ('Original Sphere Object = ', S)
print ('Rotated Sphere Object = ', S_2)

```

```

u before norm: 1    + 1    + 1
u|u: 3
u normalized: 0.577    + 0.577    + 0.577
I3: 1
B (Rotation Plane): 0.577    + -0.577    + 0.577
Rotor = 0.5 + -0.5    + 0.5    + -0.5
Original Point-Object = 5
Rotated Point-object = 4.44e-16    + 5.0    + -4.44e-16    + -2.22e-16
Original Sphere Object = 1    + 3    + 4.0    + 5.0
Rotated Sphere Object = -2.22e-16    + 1.0    + 3.0    + 4.0    + 5.0    +
2.22e-16    + 2.22e-16

```

```

[104]: # Visualize rotation
sc = GanjaScene()
# Axis of rotation (line)
axis_line = up(4*u) ^ eo ^ e1f # Line through origin and 4*u
sc.add_objects([get_coefs(axis_line)], color=Color.BLUE)

# Original objects
sc.add_objects([get_coefs(up(O)), get_coefs(S)], color=Color.RED)

```

```
# Rotated objects
sc.add_objects([get_coefs(up(0_2)), get_coefs(S_2)], color=Color.GREEN)

draw(sc, scale=0.2)
```

<IPython.core.display.Javascript object>

```
[105]: # Example 2: Rotation in a plane
phi = 2*np.pi/3 # rotation angle
b = e1*e2 # Rotation plane (xy-plane)
# b is already normalized (e1 and e2 are orthonormal)

# R = np.exp(-b * (phi/2)) # rotor
# Use alg.exp or manual formula
try:
    R = alg.exp(-b * (phi/2))
except:
    R = np.cos(phi/2) - b * np.sin(phi/2)

a = e1 + e3 # vector (1,0,1)
a_rot = R * a * ~R
print('Rotated vector = ', a_rot)
```

Rotated vector = -0.5 + 0.866 + 1.0

## 2.15 15. Dilations

Go to Top

Here is the Dilation formula in CGA :

$$D = 1 + \frac{1-d}{1+d} e_{\infty} \wedge e_o,$$

where  $d$  is the scale factor, i.e.,  $d = 1$  will leave the object as it is.

**Remark** The scaling is done with respect to the origin  $e_o$ .

```
[106]: # Dilations
scale_factor = 0.5
E = eo ^ einf
# D = np.exp(-0.5 * np.log(scale_factor) * E)
try:
    D = alg.exp(-0.5 * np.log(scale_factor) * E)
except:
    # Manual expansion for dilation: D = cosh(alpha/2) - sinh(alpha/2) E ?
    # No, E*E = (eo^einf)*(eo^einf) = eo*einf*eo*einf = -eo*eo*einf*einf = 0?
    # eo^2=0, einf^2=0.
    # (eo^einf)^2 = eo einf eo einf = - eo eo einf einf = 0.
    # So exp(x E) = 1 + x E.
    # D = 1 - 0.5 * ln(d) * E
```

```

# Let's check the formula  $D = 1 + (1-d)/(1+d) * E$ 
# If  $d=0.5$ ,  $\ln(d) = -0.693$ .  $-0.5*\ln(d) = 0.346$ .
#  $(1-0.5)/(1+0.5) = 0.5/1.5 = 1/3 = 0.333$ .
# They are close but not identical.
# The formula  $D = 1 + (1-d)/(1+d) * E$  is for the conformal map.
# Let's use the formula provided in the markdown if possible, or the
↪ exponential one.
# The exponential form is  $D = \exp(-\ln(d)/2 * E)$ .
# Since  $E^2 = 1$  (wait, let's check  $E^2$ )
#  $eo = 0.5(e5-e4)$ ,  $einf = e5+e4$ .
#  $eo.einf = -1$ .
#  $E = eo^einf = eo*einf - eo.einf = eo*einf + 1$ .
#  $E * E = (eo*einf + 1)(eo*einf + 1) \dots$ 
# Actually, let's just use the formula from the markdown:
#  $D = 1 + ((1-d)/(1+d)) * E$ 
D = 1 + ((1-scale_factor)/(1+scale_factor)) * E

print('Dilator D = ', D)

```

Dilator D = 1 + -0.333

```

[107]: # Apply dilation
S = up(e1 + 3*e2) - 0.5 * 1**2 * einf
S_3 = D * S * ~D

print('Original Sphere = ', S)
print('Dilated Sphere = ', S_3)

sc = GanjaScene()
sc.add_objects([get_coefs(S)], color=Color.RED)
sc.add_objects([get_coefs(S_3)], color=Color.GREEN)
draw(sc, scale=0.25)

```

Original Sphere = 1 + 3 + 4.0 + 5.0  
Dilated Sphere = 0.889 + 2.67 + 7.78 + 8.22

<IPython.core.display.Javascript object>

## 2.16 16. Motors

Go to Top

```

[108]: # Motors (Rotation + Translation)
# Create a translator T
t = 2*e1 # Translation vector (2,0,0)
T = 1 - 0.5 * t * einf # Translator  $T = 1 - t*einf/2$ 
# Create a rotor R (from previous example)
# R was rotation by 120 deg around (1,1,1)

```

```
M = T * R # Motor
print('Motor M = ', M)
```

Motor M = 0.5 + -0.866 + -0.5 + -0.5 + 0.866 + 0.866

```
[109]: # Apply motor
S = up(e1 + 3*e2) - 0.5 * 1**2 * e1f
S_4 = M * S * ~M

print('Original Sphere = ', S)
print('Transformed Sphere = ', S_4)

sc = GanjaScene()
sc.add_objects([get_coefs(S)], color=Color.RED)
sc.add_objects([get_coefs(S_4)], color=Color.GREEN)
draw(sc, scale=0.25)
```

Original Sphere = 1 + 3 + 4.0 + 5.0  
 Transformed Sphere = -1.1 + -0.634 + -0.196 + 0.804 + -5.55e-16  
 + -5.55e-16 + 1.11e-16

<IPython.core.display.Javascript object>

## 2.17 17. Rotating lines & Circles

Go to Top

```
[110]: # Rotating lines
# Define two lines
# Line 1: x-axis
line1 = (up(0*e1) ^ up(e1) ^ e1f)
line1 = line1 / np.sqrt(get_scalar(line1|line1)) # Normalize

# Line 2: y-axis
line2 = (up(0*e1) ^ up(e2) ^ e1f)
# line2 = line2.normalized()

print ('line1 ', line1)
print ('line2 ', line2)

# Rotor that rotates line1 to line2
# R_z = np.exp(-(e1^e2) * (np.pi/4))
try:
    R_z = alg.exp(-(e1^e2) * (np.pi/4))
except:
    R_z = np.cos(np.pi/4) - (e1^e2) * np.sin(np.pi/4)

print('R_z ', R_z)
```



```
line1_rot = R_z * line1 * ~R_z
print('Rotated line1 = ', line1_rot)
```

```
line1  1.0
line2  1.0
R_z    0.707 + -0.707j
Rotated line1 =  2.22e-16 + 1.0j
```

```
[111]: # Rotate circles
# Circle 1: Unit circle in xy-plane
p1 = up(e1)
p2 = up(e2)
p3 = up(-e1)
circle1 = p1 ^ p2 ^ p3
print('circle1 ', circle1)

# Rotate using R_z (90 deg around z)
# R_z defined in previous cell, but let's redefine to be safe or use it if
↳ available
try:
    R_z = alg.exp(-(e1^e2) * (np.pi/4))
except:
    R_z = np.cos(np.pi/4) - (e1^e2) * np.sin(np.pi/4)

circle1_rot = R_z * circle1 * ~R_z
print('Rotated circle1 = ', circle1_rot)

# Let's rotate around x-axis by 90 degrees
# R_x = np.exp(-(e2^e3) * (np.pi/4))
try:
    R_x = alg.exp(-(e2^e3) * (np.pi/4))
except:
    R_x = np.cos(np.pi/4) - (e2^e3) * np.sin(np.pi/4)

circle1_rot_x = R_x * circle1 * ~R_x
print('Rotated circle1 around x = ', circle1_rot_x)

sc = GanjaScene()
sc.add_objects([get_coefs(circle1)], color=Color.BLUE)
sc.add_objects([get_coefs(circle1_rot_x)], color=Color.RED)
draw(sc, scale=0.25)
```

```
circle1  2.0
Rotated circle1 =  2.0
Rotated circle1 around x =  4.44e-16 + 2.0j
```

<IPython.core.display.Javascript object>